

Interpretability in Large Language Models

Gabriel Reis

Lucas Perez

Rafael Martins

Dictionary Learning

Word Embedding Visualization Via Dictionary Learning

Juexiao Zhang*^{1,2} **Yubei Chen***^{1,3} **Brian Cheung**^{1,3} **Bruno A Olshausen**^{1,3,4}

¹ Berkeley AI Research (BAIR), UC Berkeley

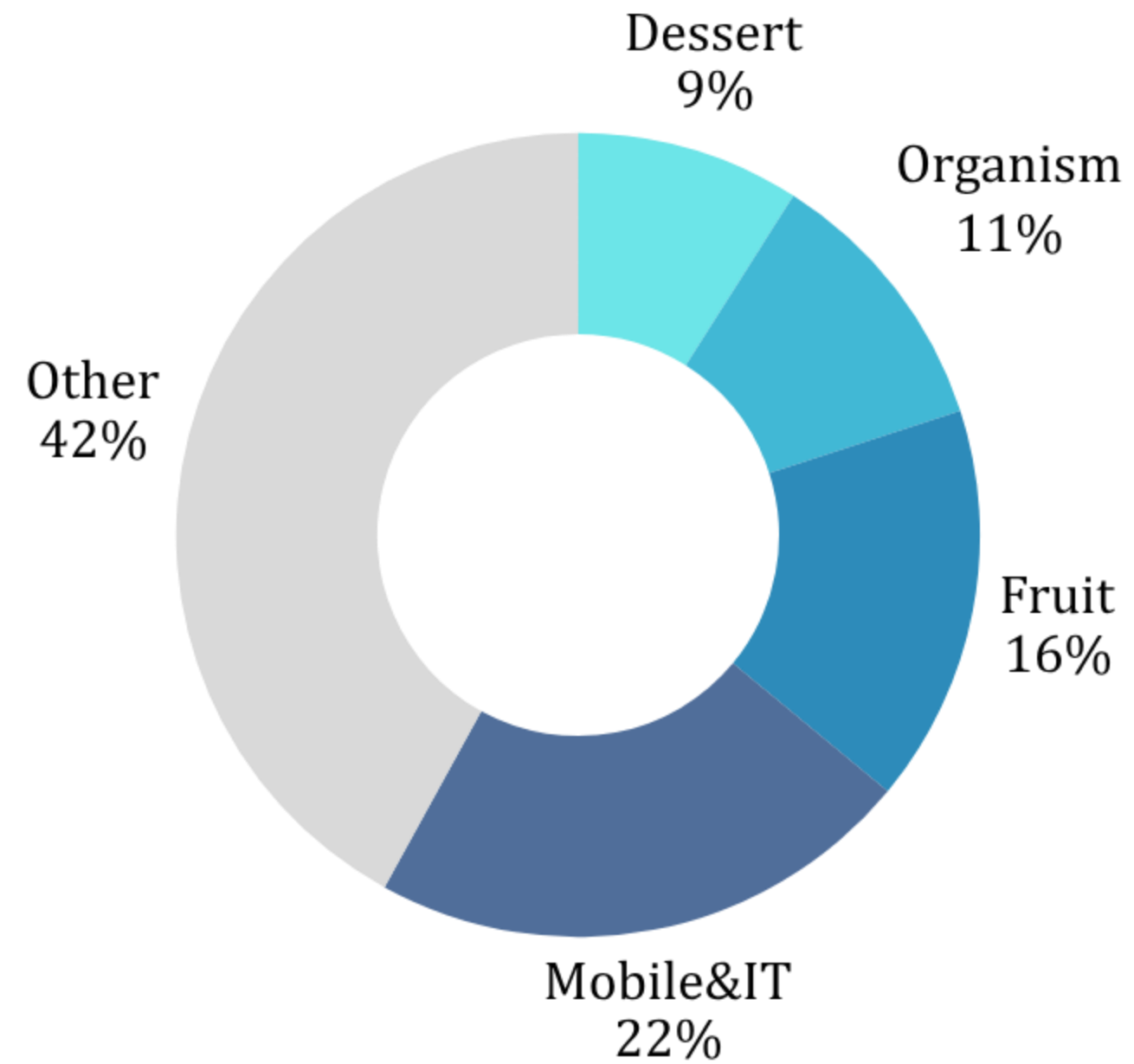
² Department of Electrical Engineering, Tsinghua University, Beijing, China

³ Redwood Center for Theoretical Neuroscience, UC Berkeley

⁴ Helen Wills Neuroscience Institute & School of Optometry, UC Berkeley

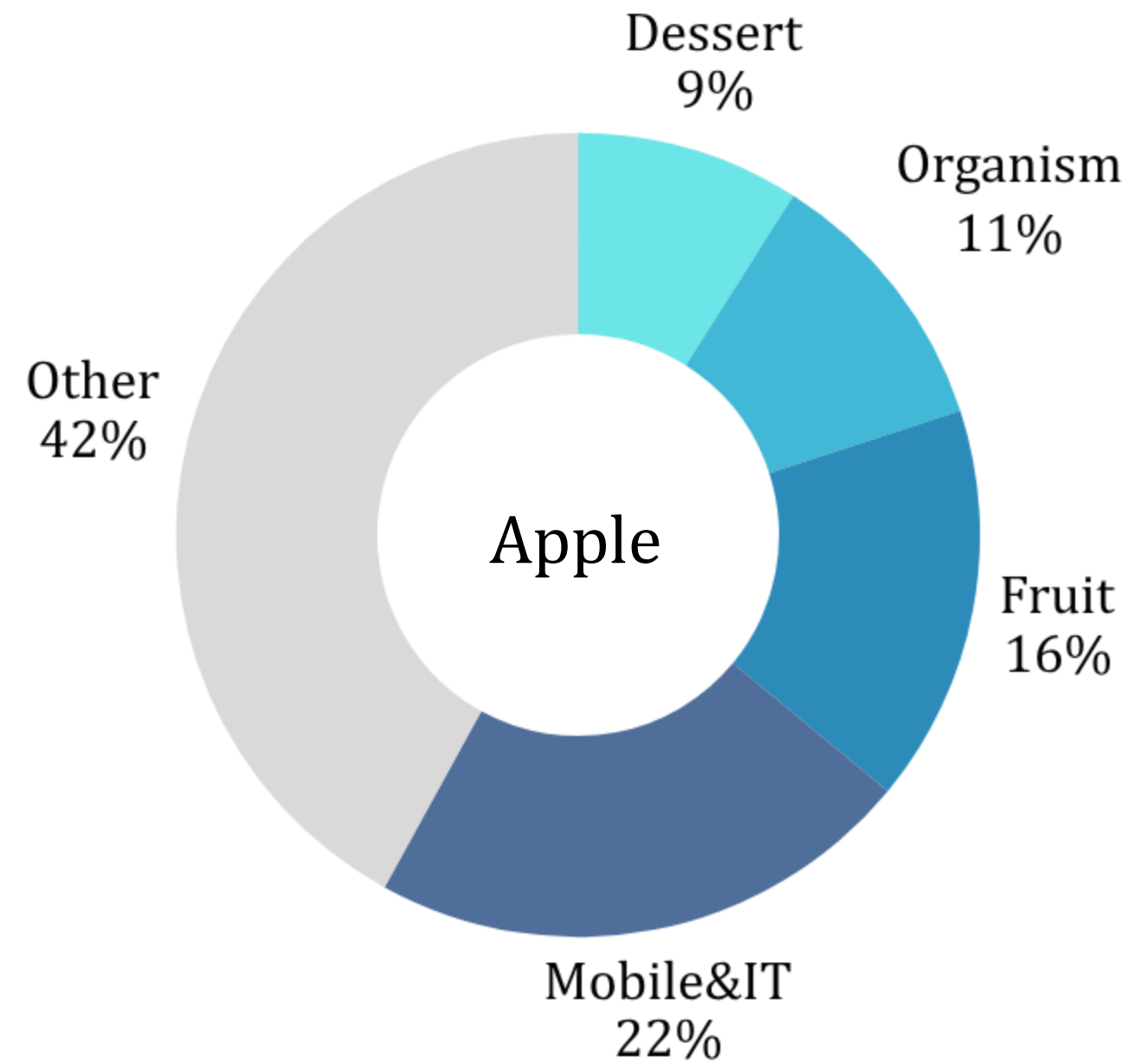
Decomposition

Ultimately, words can be decomposed into factors



Decomposition

Ultimately, words can be decomposed into factors



Factors

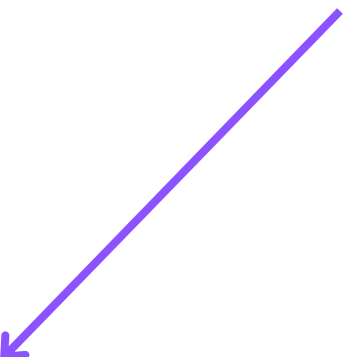
Main idea: "Word vectors can be obtained by **linear superposition** of sparse **word factors**"

$$\boldsymbol{x} = \Phi \boldsymbol{\alpha} + \boldsymbol{\epsilon}, \text{ s.t. } \boldsymbol{\alpha} \succeq 0,$$

Factors

Main idea: "Word vectors can be obtained by **linear superposition** of sparse **word factors**"

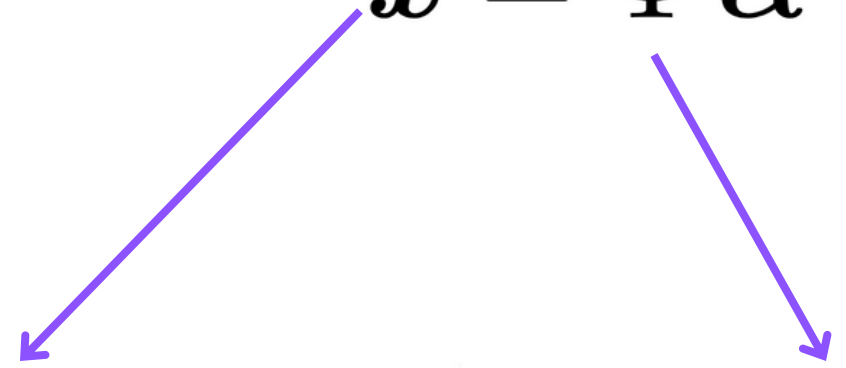
$$\mathbf{x} = \Phi \boldsymbol{\alpha} + \boldsymbol{\epsilon}, \text{ s.t. } \boldsymbol{\alpha} \succeq 0,$$


$$\begin{pmatrix} 1.23 \\ 0.12 \\ \dots \\ 0.76 \end{pmatrix}$$

embedding
n=300

Factors

Main idea: "Word vectors can be obtained by **linear superposition** of sparse **word factors**"

$$\mathbf{x} = \Phi \boldsymbol{\alpha} + \boldsymbol{\epsilon}, \text{ s.t. } \boldsymbol{\alpha} \succeq 0,$$

$$\begin{pmatrix} 1.23 \\ 0.12 \\ \dots \\ 0.76 \end{pmatrix} = \begin{pmatrix} 1.1 & 1.2 & 0.3 & 0.1 & \dots & 0.8 \\ 0.4 & 0.8 & 1.2 & 0.4 & \dots & 1.1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0.9 & 0.9 & 0.1 & 0.5 & \dots & 0.5 \end{pmatrix}$$

embedding
n=300

factors
n x d

Factors

Main idea: "Word vectors can be obtained by **linear superposition** of sparse **word factors**"

$$\mathbf{x} = \Phi \boldsymbol{\alpha} + \boldsymbol{\epsilon}, \text{ s.t. } \boldsymbol{\alpha} \succeq 0,$$

$$\begin{pmatrix} 1.23 \\ 0.12 \\ \dots \\ 0.76 \end{pmatrix} = \begin{pmatrix} 1.1 & 1.2 & 0.3 & 0.1 & \dots & 0.8 \\ 0.4 & 0.8 & 1.2 & 0.4 & \dots & 1.1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0.9 & 0.9 & 0.1 & 0.5 & \dots & 0.5 \end{pmatrix}$$

factors
n x d

Factors

Main idea: "Word vectors can be obtained by **linear superposition** of sparse **word factors**"

$$\mathbf{x} = \Phi \boldsymbol{\alpha} + \boldsymbol{\epsilon}, \text{ s.t. } \boldsymbol{\alpha} \succeq 0,$$
$$\begin{pmatrix} 1.23 \\ 0.12 \\ \dots \\ 0.76 \end{pmatrix} = \begin{pmatrix} 1.1 & 1.2 & 0.3 & 0.1 & \dots & 0.8 \\ 0.4 & 0.8 & 1.2 & 0.4 & \dots & 1.1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0.9 & 0.9 & 0.1 & 0.5 & \dots & 0.5 \end{pmatrix} \begin{pmatrix} 1 \\ 2.1 \\ 1.1 \\ 0.3 \\ \dots \\ 0.8 \end{pmatrix}$$

embedding
n=300

factors
n x d

factor weights
d x 1

Factors

Main idea: "Word vectors can be obtained by **linear superposition** of sparse **word factors**"

$$\mathbf{x} = \Phi \boldsymbol{\alpha} + \boldsymbol{\epsilon}, \text{ s.t. } \boldsymbol{\alpha} \succeq 0,$$

$$\begin{matrix} \text{n} \end{matrix} \begin{pmatrix} 1.23 \\ 0.12 \\ \dots \\ 0.76 \end{pmatrix} = \begin{matrix} \text{d} \end{matrix} \begin{pmatrix} 1.1 & 1.2 & 0.3 & 0.1 & \dots & 0.8 \\ 0.4 & 0.8 & 1.2 & 0.4 & \dots & 1.1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0.9 & 0.9 & 0.1 & 0.5 & \dots & 0.5 \end{pmatrix} \begin{pmatrix} 1 \\ 2.1 \\ 1.1 \\ 0.3 \\ \dots \\ 0.8 \end{pmatrix}$$

$$\mathbf{d} > \mathbf{n}$$

Spectral Clustering

In practice, the matrix usually contains **more factors than semantic meaning**:

- overcomplete sparse coding
- manifolds (lower dimensional curve in a high dimensional space)

$$\begin{matrix} n \\ \left| \right. \end{matrix} \begin{pmatrix} 1.23 \\ 0.12 \\ \dots \\ 0.76 \end{pmatrix} = \begin{pmatrix} \boxed{1.1} & 1.2 & 0.3 & \boxed{0.1} & \dots & 0.8 \\ \boxed{0.4} & 0.8 & 1.2 & \boxed{0.4} & \dots & 1.1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ \boxed{0.9} & 0.9 & 0.1 & \boxed{0.5} & \dots & 0.5 \end{pmatrix} \begin{pmatrix} 1 \\ 2.1 \\ 1.1 \\ 0.3 \\ \dots \\ 0.8 \end{pmatrix}$$

Fruit Fruit

Spectral Clustering

In practice, the matrix usually contains **more factors than semantic meaning**:

- **overcomplete sparse** coding
- **manifolds** (lower dimensional curve in a high dimensional space)

↙ Spectral
Clustering

$$n \begin{pmatrix} 1.23 \\ 0.12 \\ \dots \\ 0.76 \end{pmatrix} = \begin{pmatrix} 1.1 & 1.2 & 0.3 & 0.1 & \dots & 0.8 \\ 0.4 & 0.8 & 1.2 & 0.4 & \dots & 1.1 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0.9 & 0.9 & 0.1 & 0.5 & \dots & 0.5 \end{pmatrix} \begin{pmatrix} 1 \\ 2.1 \\ 1.1 \\ 0.3 \\ \dots \\ 0.8 \end{pmatrix}$$

Spectral Clustering

In practice, the matrix usually contains **more columns than semantic meaning**:

- **overcomplete sparse** coding
- **manifolds** (lower dimensional curve in a high dimensional space)

↙ Spectral
Clustering

$$n \begin{pmatrix} 1.23 \\ 0.12 \\ \dots \\ 0.76 \end{pmatrix} = \begin{pmatrix} \begin{matrix} 1.1 \\ 0.4 \\ \dots \\ 0.9 \end{matrix} & 1.2 & \begin{matrix} 0.3 \\ 1.2 \\ \dots \\ 0.1 \end{matrix} & \begin{matrix} 0.1 \\ 0.4 \\ \dots \\ 0.5 \end{matrix} & \dots & \begin{matrix} 0.8 \\ 1.1 \\ \dots \\ 0.5 \end{matrix} \end{pmatrix} \begin{pmatrix} 1 \\ 2.1 \\ 1.1 \\ 0.3 \\ \dots \\ 0.8 \end{pmatrix}$$

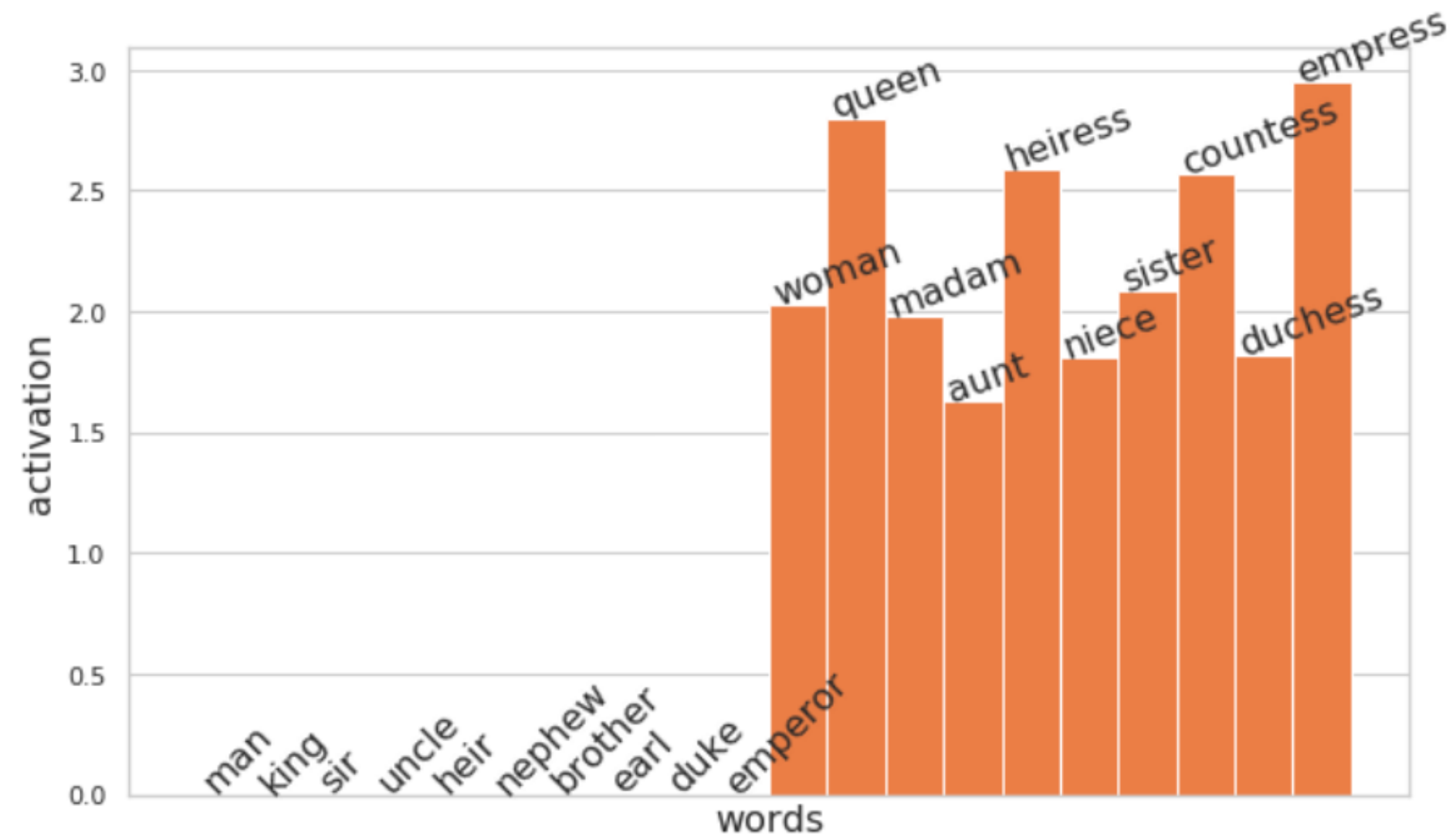
Fruit Mobile
& IT

The diagram shows a matrix factorization equation. On the left, a vertical vector of size 'n' is multiplied by a matrix. This matrix is equal to the product of two matrices. The first matrix has columns highlighted in light purple and light yellow. Above the purple columns is the label 'Fruit' with a purple arrow pointing to the first column. Above the yellow columns is the label 'Mobile & IT' with a yellow arrow pointing to the third column. A purple arrow also points from the word 'manifolds' in the list above to the purple columns. A yellow arrow points from the word 'overcomplete' in the list above to the yellow columns. The second matrix has a single column of values. The overall equation is: $n \begin{pmatrix} 1.23 \\ 0.12 \\ \dots \\ 0.76 \end{pmatrix} = \begin{pmatrix} \begin{matrix} 1.1 \\ 0.4 \\ \dots \\ 0.9 \end{matrix} & 1.2 & \begin{matrix} 0.3 \\ 1.2 \\ \dots \\ 0.1 \end{matrix} & \begin{matrix} 0.1 \\ 0.4 \\ \dots \\ 0.5 \end{matrix} & \dots & \begin{matrix} 0.8 \\ 1.1 \\ \dots \\ 0.5 \end{matrix} \end{pmatrix} \begin{pmatrix} 1 \\ 2.1 \\ 1.1 \\ 0.3 \\ \dots \\ 0.8 \end{pmatrix}$

Naming Clusters

By analyzing the top-k words in each cluster, they were named.

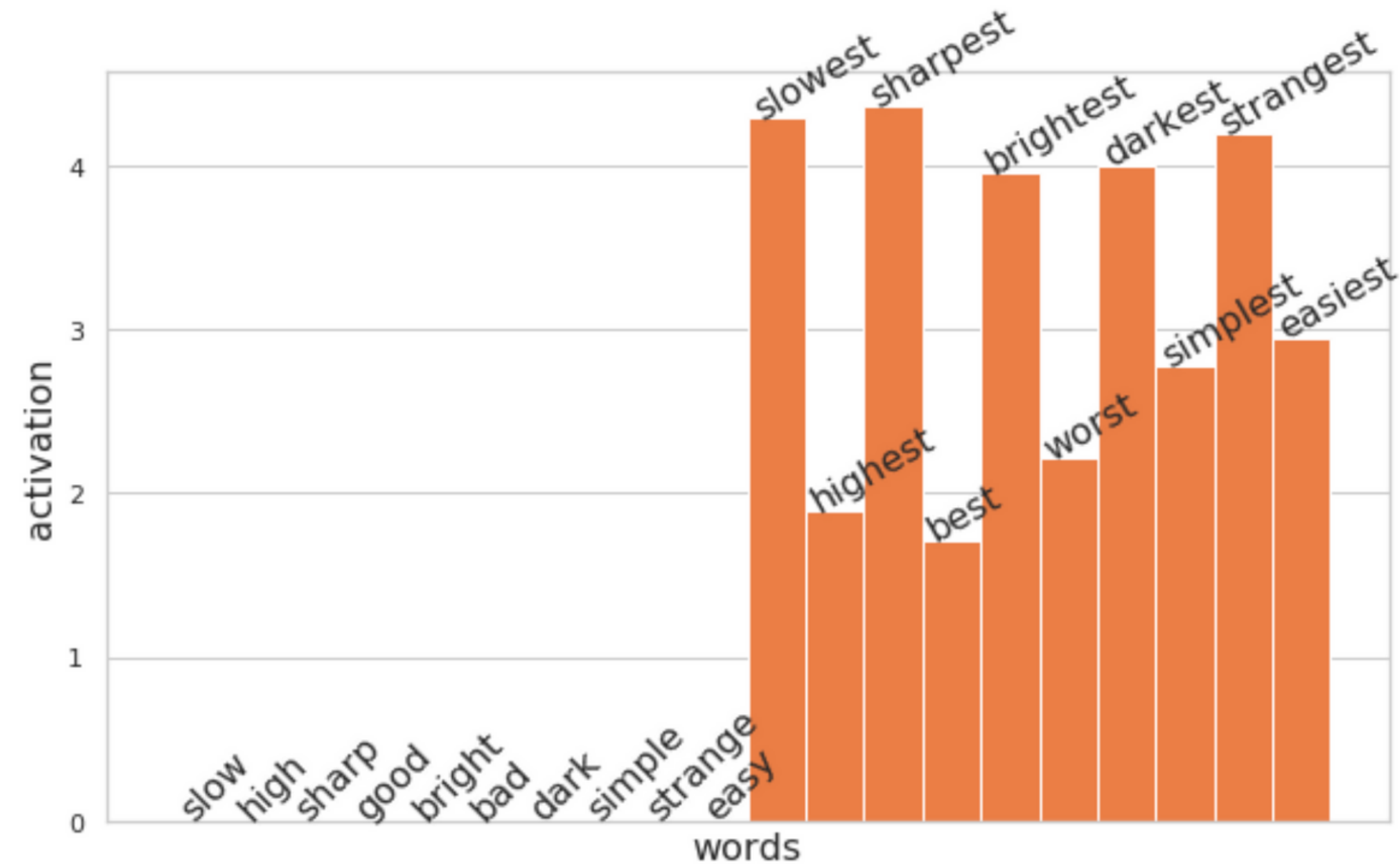
Example: **"Female" cluster**



Naming Clusters

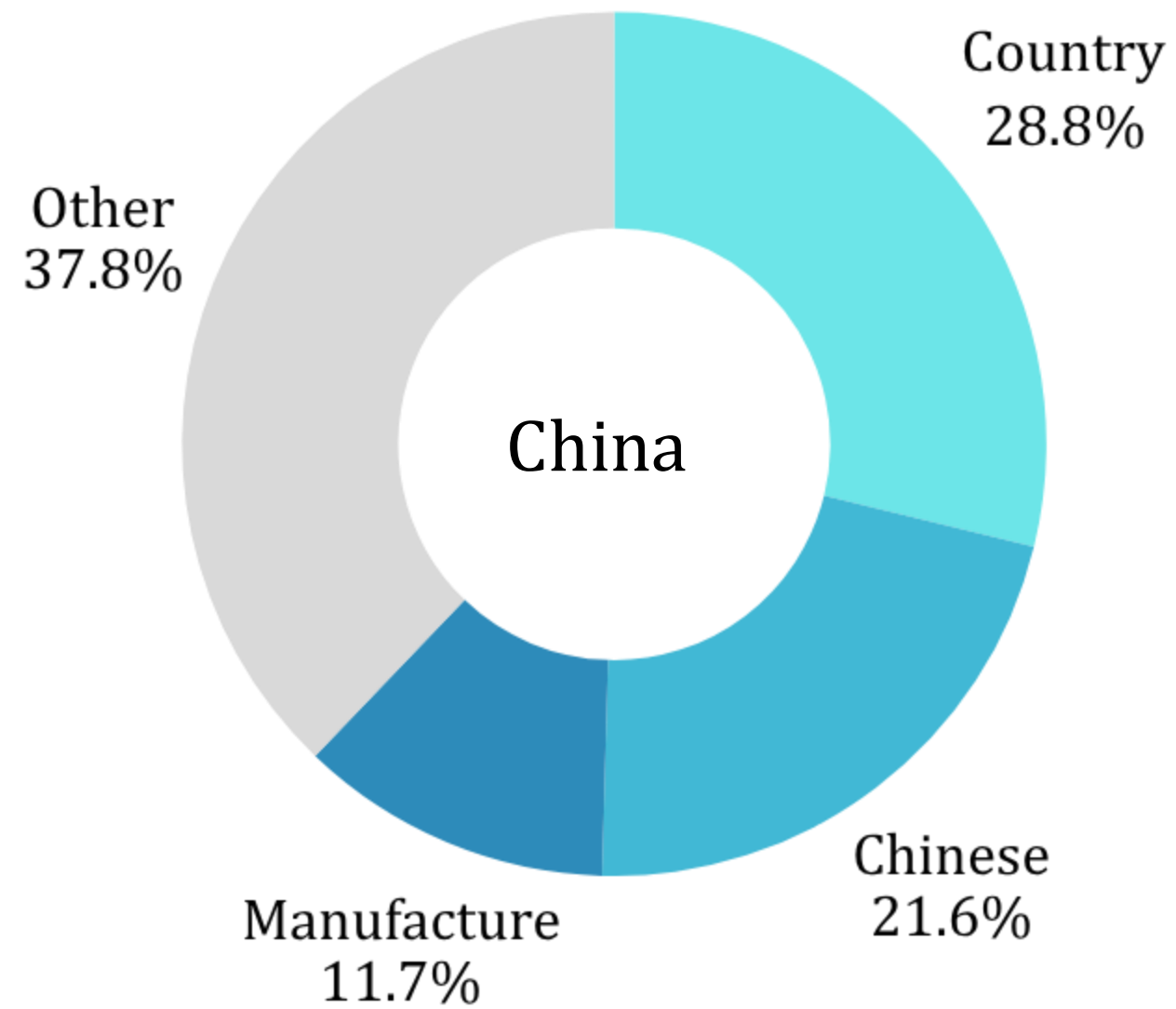
By analyzing the top-k words in each cluster, they were named.

Example: **"Superlative" cluster**



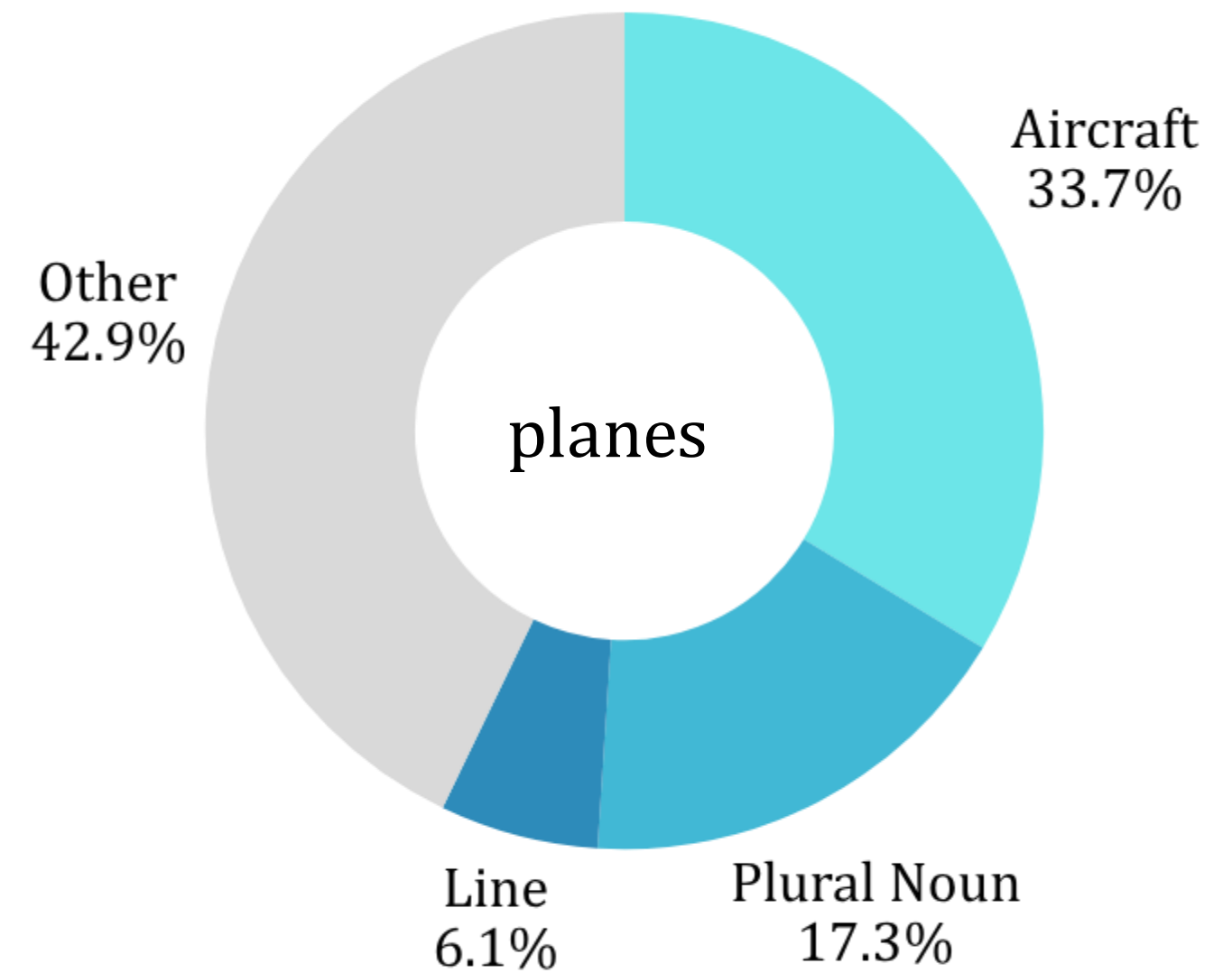
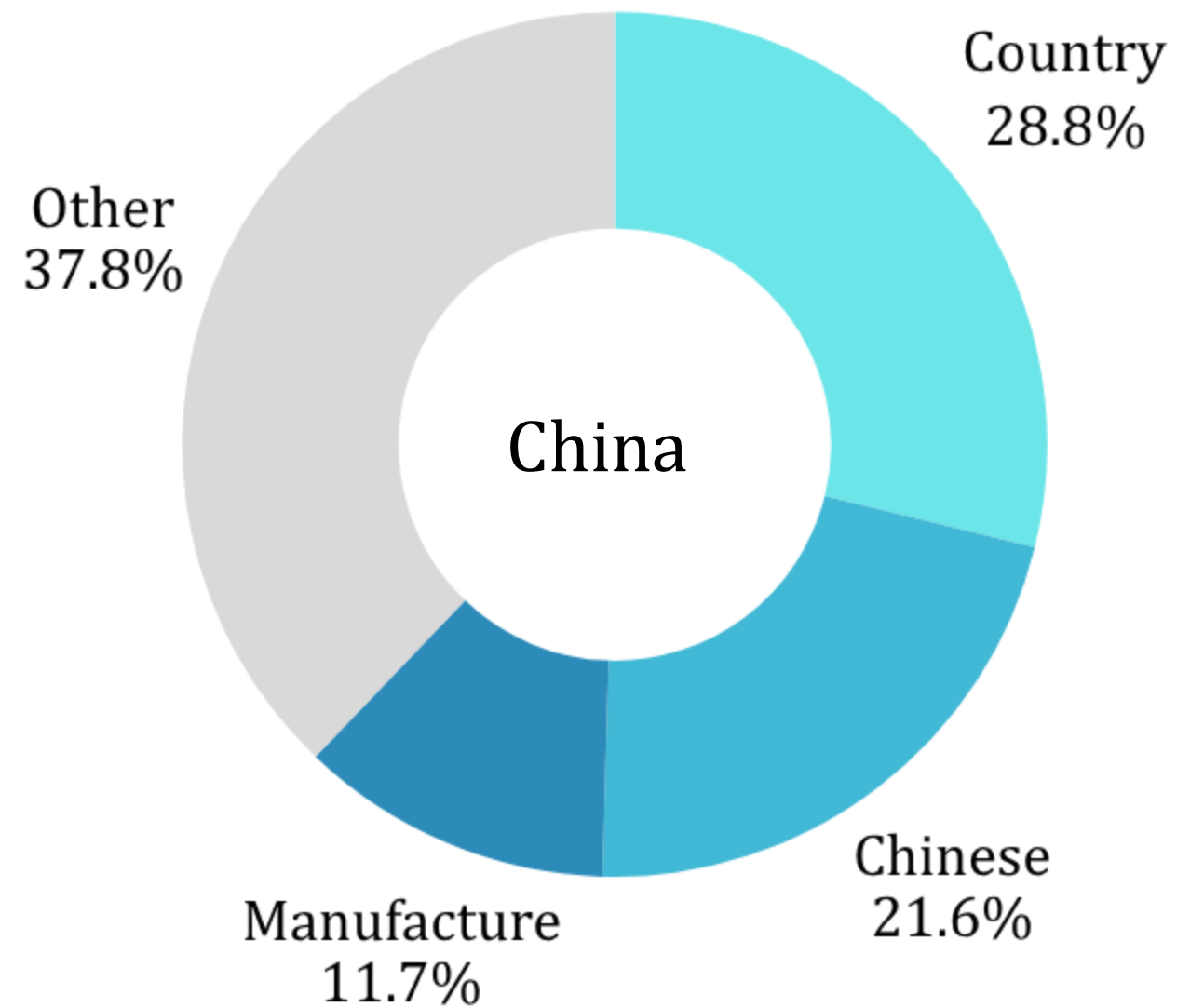
Decomposing Words

Word decomposition to understand the clusters



Decomposing Words

Word decomposition to understand the clusters



Vector + Factor

Adding **vectors** together to asses **word analogy** is a common practice

$$V_{man} - V_{woman} = V_{king} - V_{queen}$$

Vector + Factor

Adding **vectors** together to asses **word analogy** is a common practice

$$V_{man} - V_{woman} = V_{king} - V_{queen}$$

Adding **vectors and factors** add layers to our understanding of embeddings

$$V_{good} + F_{superlative} = V_{best}$$

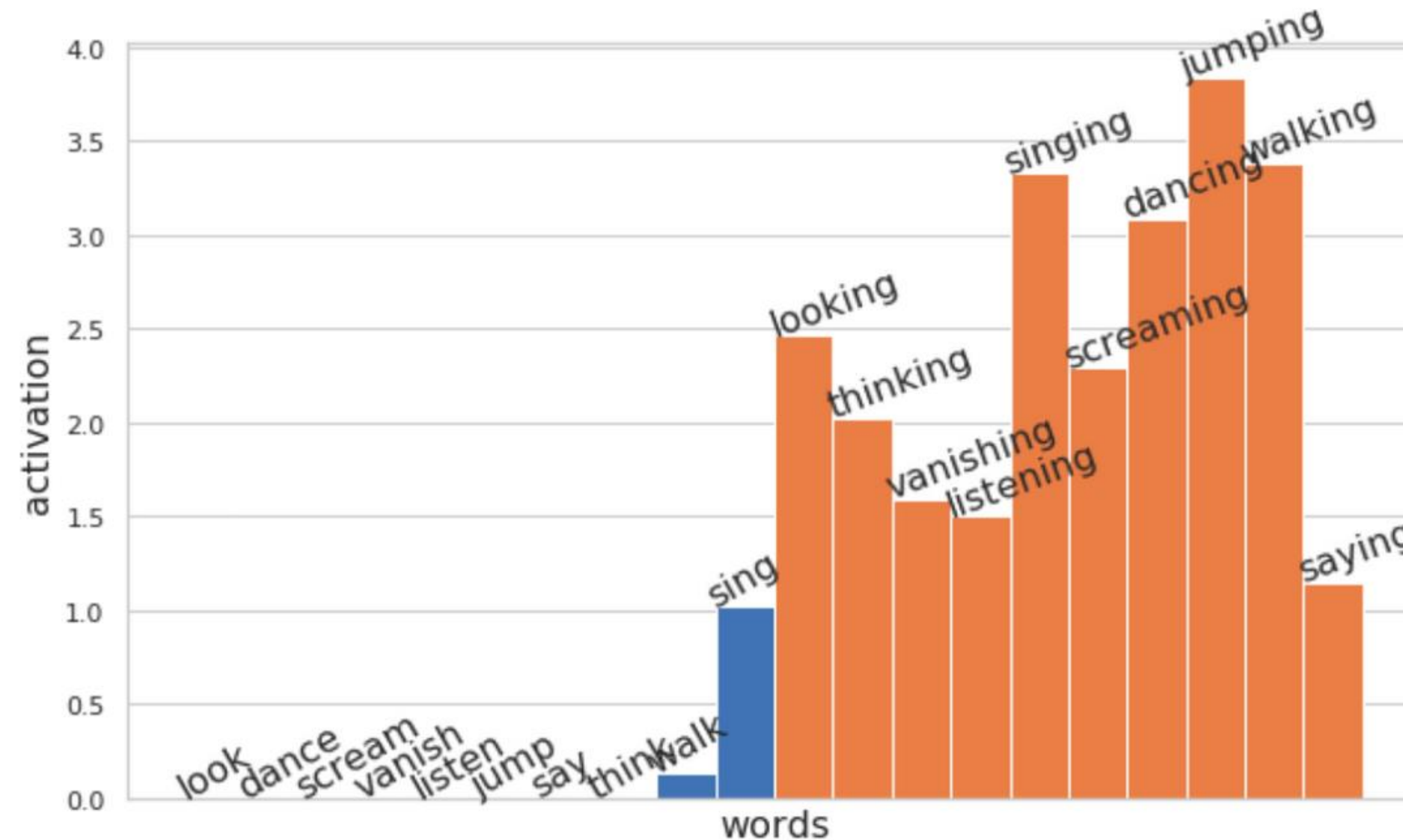
$$V_{hospital} + F_{vehicle} = V_{ambulance}$$

$$V_{kindle} - F_{book} = V_{ipad}$$

Different Embeddings = Different Factors

Different **training models** yield different results.

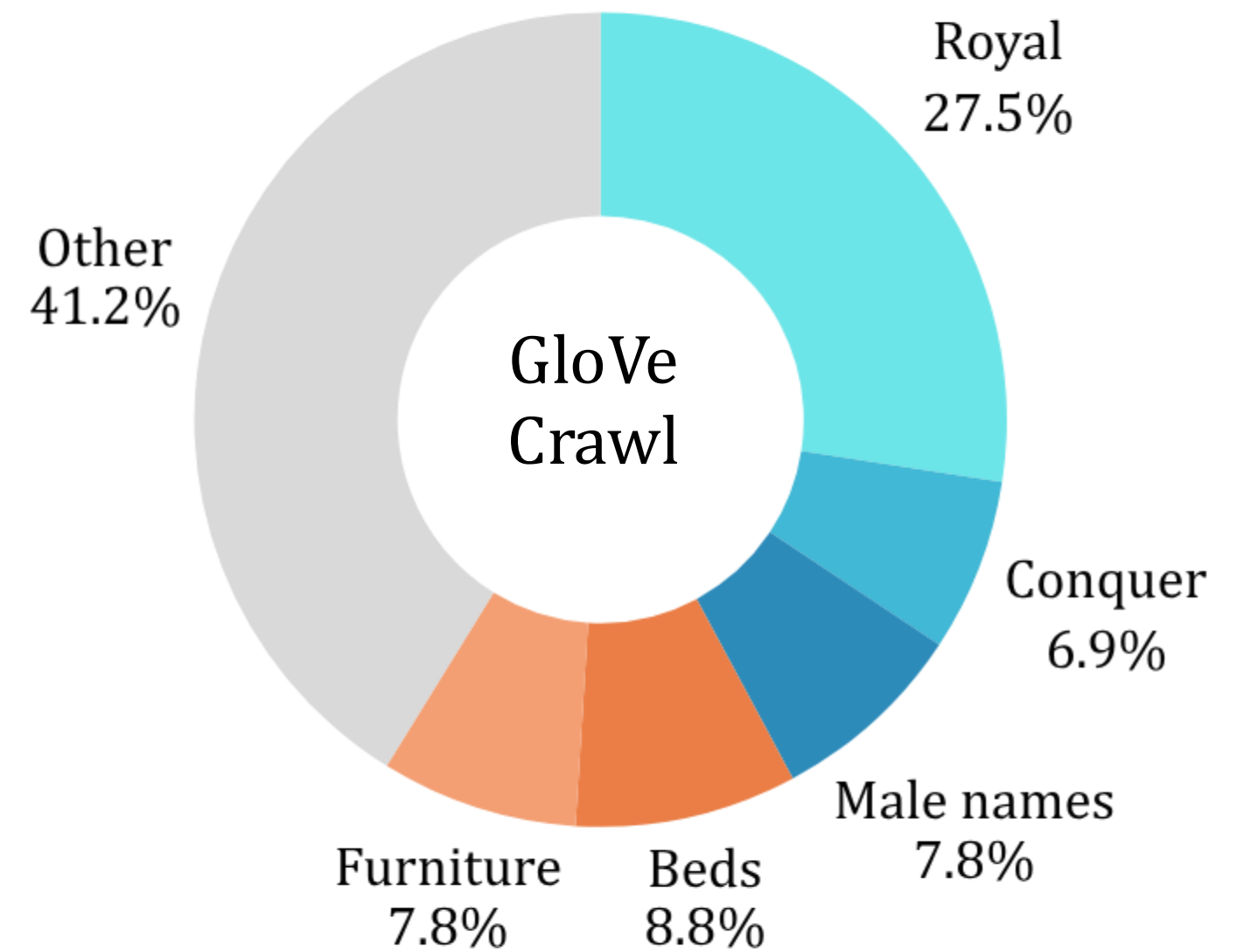
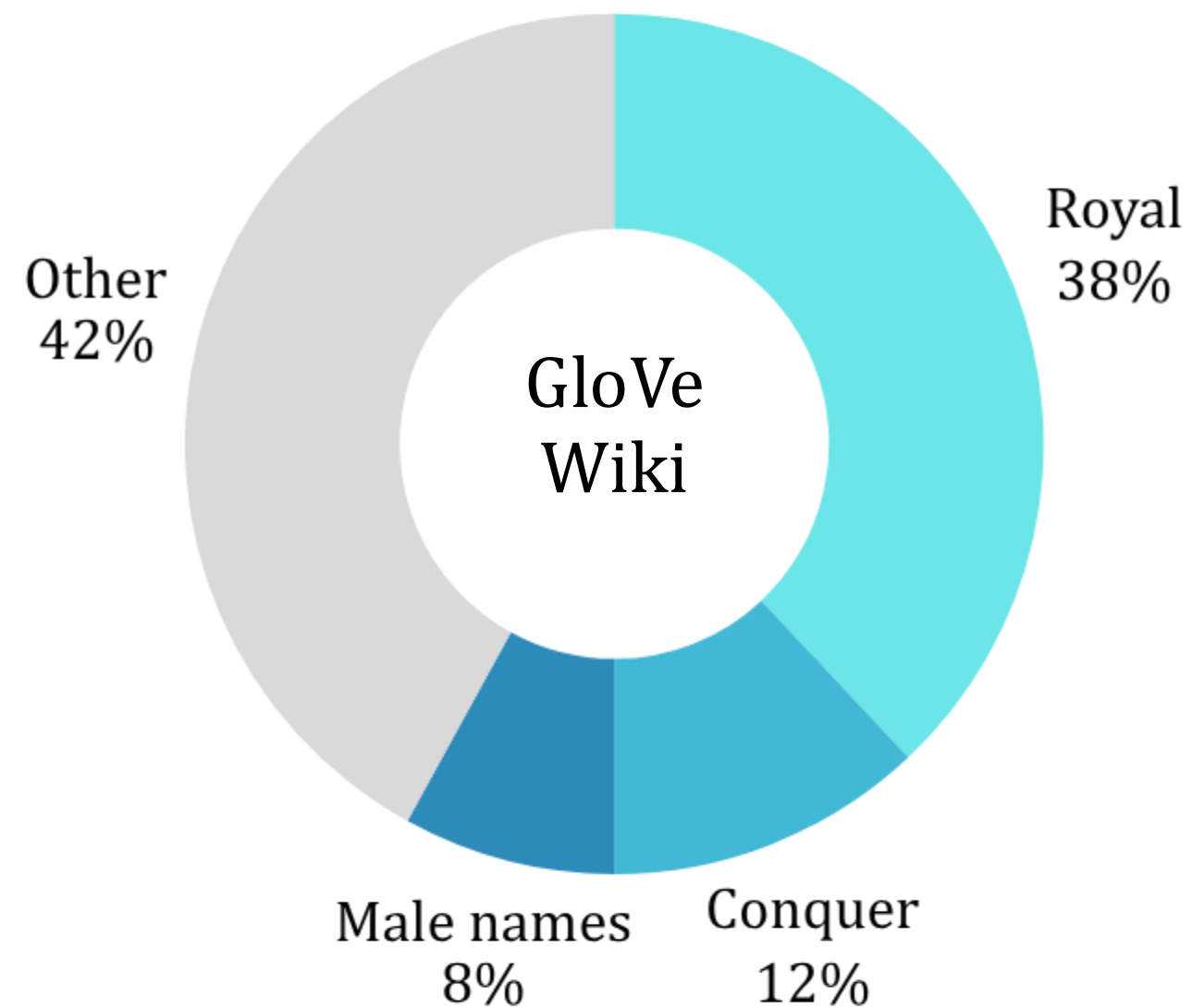
Example: fastText uses **tri-grams**, so "sing" has high activation among **gerunds**



Different Embeddings = Different Factors

Same training with **different data** generates different factor weights

Example: different factor weights for the word "**king**"



Sparse Autoencoders

SPARSE AUTOENCODERS FIND HIGHLY INTER- PRETABLE FEATURES IN LANGUAGE MODELS

Hoagy Cunningham^{*12}, Aidan Ewart^{*13}, Logan Riggs^{*1}, Robert Huben, Lee Sharkey⁴

¹EleutherAI, ²MATS, ³Bristol AI Safety Centre, ⁴Apollo Research

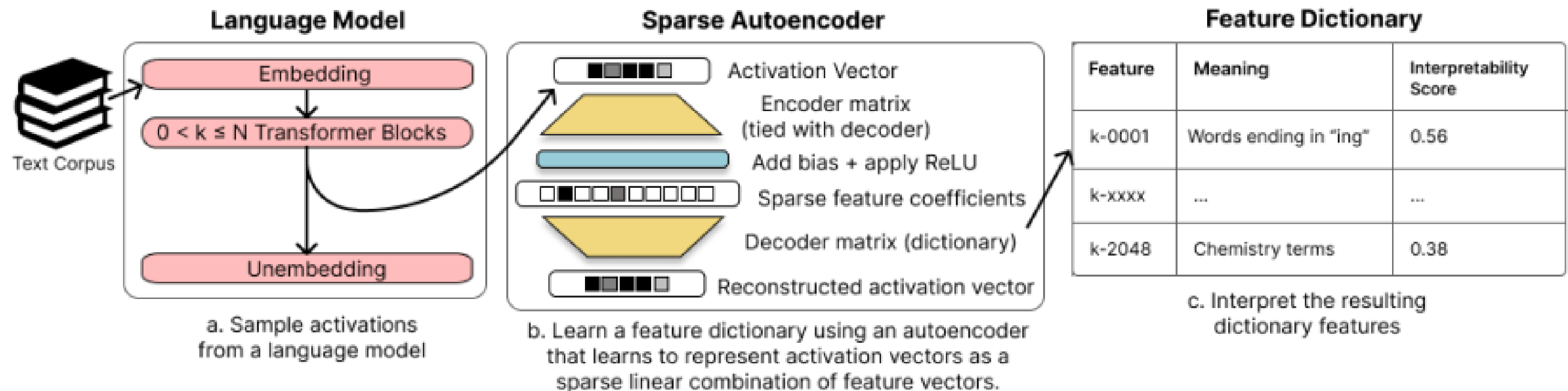
{hoagycunningham, aidanprattewart, logansmith5}@gmail.com

Motivation

- The paper focuses on dictionary learning of LLM activations. Same idea but the data are hidden states inside a transformer, not inputs.
- Superposition hurts interpretability: Many unrelated concepts share neurons/directions → activations are entangled → single units are hard to explain
- We would like to find a representation where each meaningful concept corresponds to a single feature direction that activates rarely and specifically (monosemanticity).
- Features become nameable (“apostrophe feature”, “parentheses feature”) and surgically editable, giving us handles to test causal hypotheses about model behavior

Sparse Autoencoders (SAE)

Main idea: Train a SAE on LLM activations to get a sparse dictionary of **human-interpretable features**



Why sparse? Penalizing code density pushes solutions where few features fire per token, encouraging features that align with **coherent patterns** rather than mixtures.

Sparse Autoencoders (SAE)

On activation vector $\mathbf{x}$, SAE produces the output

$$\begin{aligned}\mathbf{c} &= \text{ReLU}(M\mathbf{x} + \mathbf{b}) \\ \hat{\mathbf{x}} &= M^T \mathbf{c} \\ &= \sum_{i=0}^{d_{\text{hid}}-1} c_i \mathbf{f}_i\end{aligned}$$

and is trained to minimise the loss function

$$\mathcal{L}(\mathbf{x}) = \underbrace{||\mathbf{x} - \hat{\mathbf{x}}||_2^2}_{\text{Reconstruction loss}} + \underbrace{\alpha ||\mathbf{c}||_1}_{\text{Sparsity loss}}$$

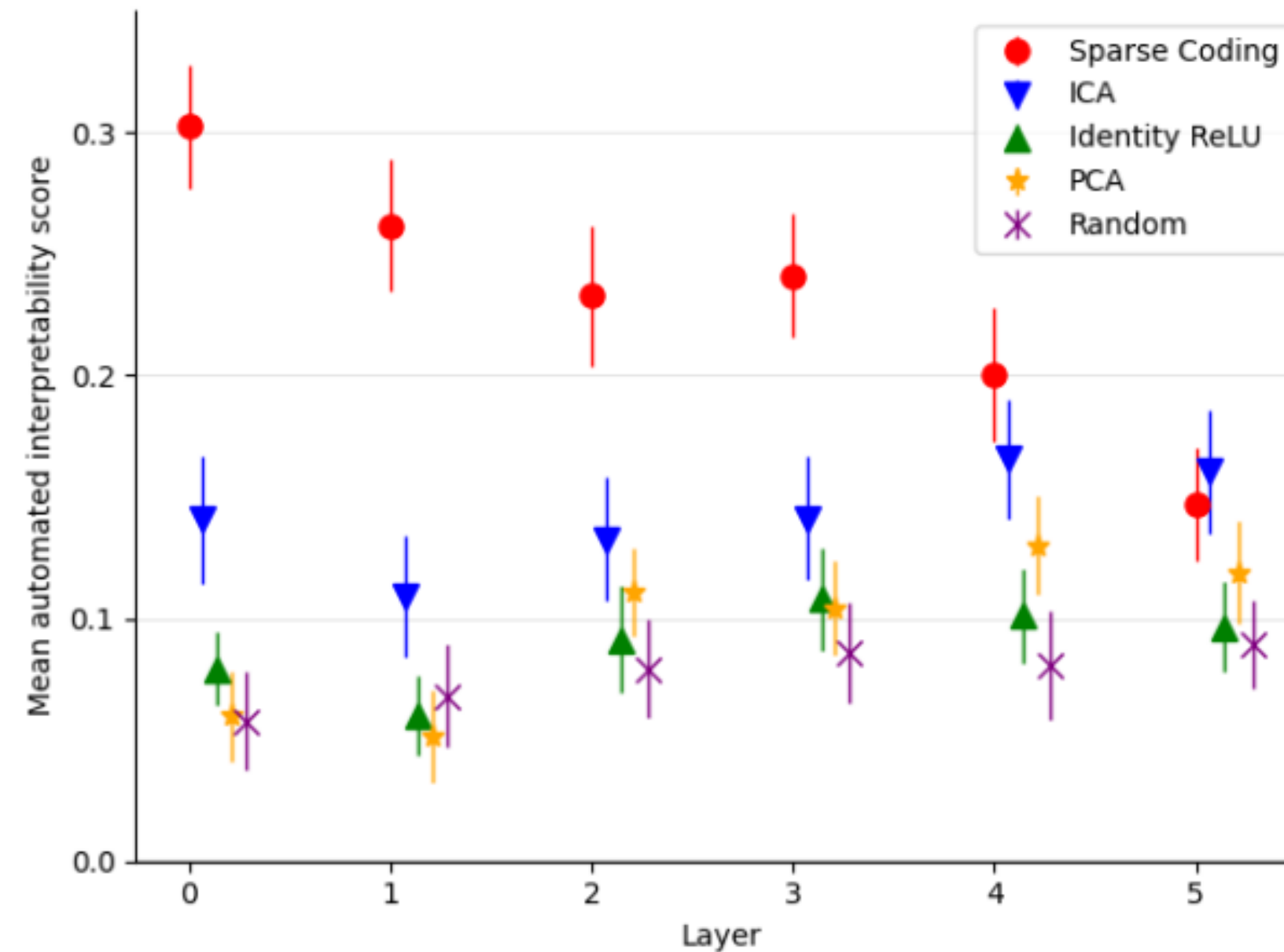
Evaluation 1: “Autointerpretability”

- **Goal**: Test whether a feature learned by SAE is human-describable and that description predicts where it will activate.
- **Procedure**:
 - a. Gather snippets where a feature fires strongly.
 - b. Use a strong LLM to propose a concise natural-language label/description.
 - c. Use that label to simulate/predict activations on held-out text.
 - d. Compute correlation (top-random) between predicted vs actual activations.

Feature	Description (Generated by GPT-4)	Interpretability Score
1-0000	parts of individual names, especially last names.	0.33
1-0001	actions performed by a subject or object.	-0.11
1-0002	instances of the letter ‘W’ and words beginning with ‘w’.	0.55
1-0003	the number ‘5’ and also records moderate to low activation for personal names and some nouns.	0.57
1-0004	legal terms and court case references.	0.19

Evaluation 1: “Autointerpretability”

- **Goal:** Test whether a feature learned by SAE is human-describable and that description predicts where it will activate.

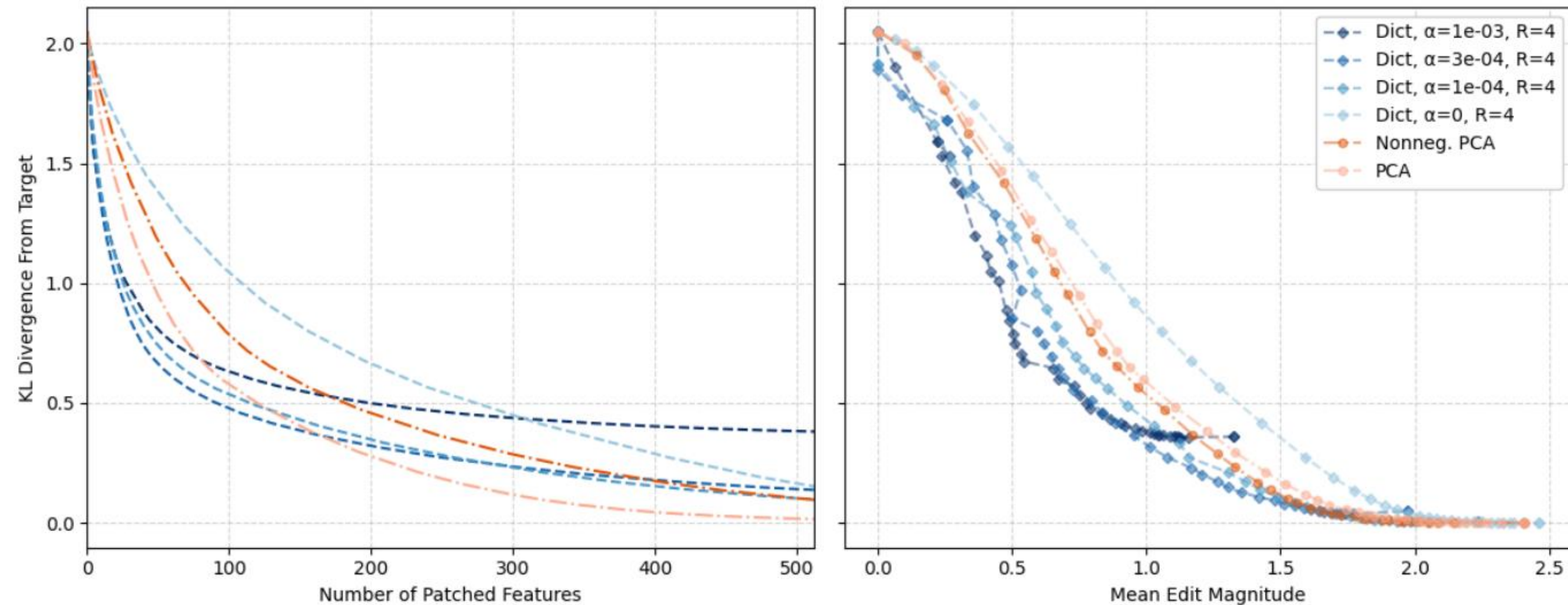


Evaluation 2: Causality via Activation Patching

- Indirect Object Identification: sentences like “ Alice and Bob went to the store. Then, Alice gave a snack to __.” The correct answer is Bob.
- Make a target version that differs in the names (e.g., replace Bob with Charlie) so that the correct answer in the target is Charlie.
- **Goal**: identify which SAE internal features determine which name the model prefers.
- Run the model twice, one for base sentence and another for the counterfactual one. Afterwards, encode both runs activations using SAE.
- Swap selected features from base to target, decode the swapped embedding and then continue the forward pass on the LLM.
- Afterwards, measure how much this intervention altered the output
 - KL between the logits of the targets input forward and the swapped

Evaluation 2: Causality via Activation Patching

- **Goal:** identify which SAE internal features determines which token the model prefers.



Attribution patching

AtP^{*}: An efficient and scalable method for localizing LLM behaviour to components

János Kramár¹, Tom Lieberum¹, Rohin Shah¹ and Neel Nanda¹

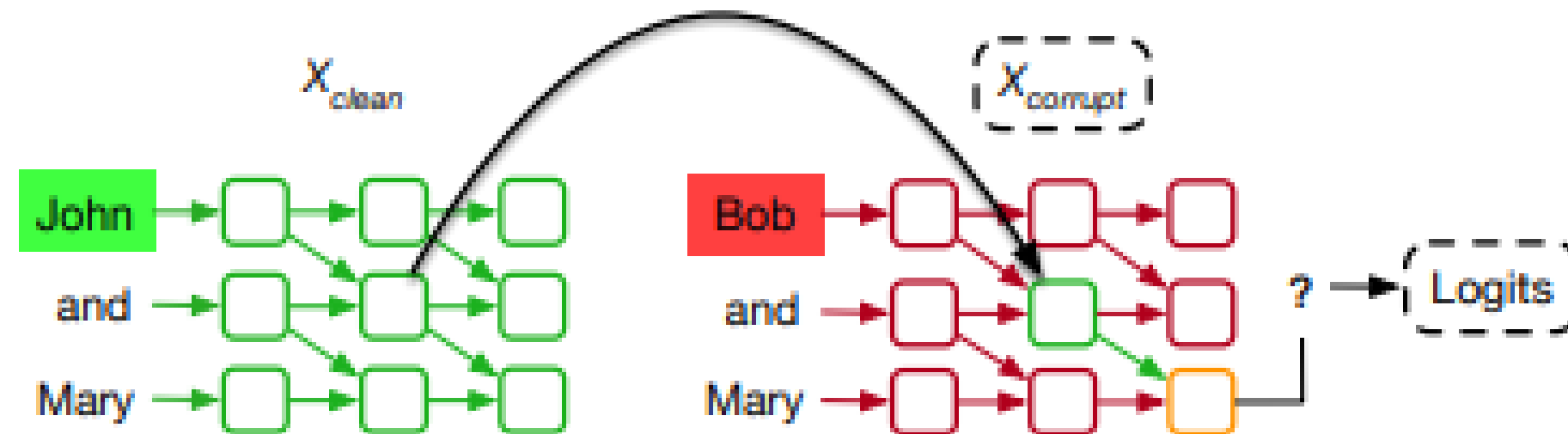
¹Google DeepMind

Motivation

- Use of mechanistic interpretability to relate behaviors to model components
- Activation patching: causal interventions on activations to test which components matter
- Problem: exhaustive activation patching is prohibitively expensive on modern LLMs
- High-level solution: Attribution Patching (AtP) for fast ranking of components; AtP* to reduce false negatives while staying scalable

Activation Patching

- Compare a clean prompt (x_{clean}) and a corrupted/noise prompt (x_{noise})
- Intervene by swapping a component's activation between clean/noise runs to see whether the model's output (or loss) is robust to the corruption



- If, despite the corruption, the output remains correct when a specific activation is patched, that component likely supports the behavior

Problem: brute-force AP is expensive

- A component's causal effect is not usually measured with only one prompt pair, but with the average across a distribution; you need a full forward pass per component per prompt pair
- In large LLMs there are millions to billions of nodes (e.g., neurons across positions)
- Total cost scales roughly like $O(|N| \cdot |D|)$ ($N \rightarrow$ nodes; $D \rightarrow$ prompt pairs)

Attribution Patching (AtP)

- Use a first-order Taylor expansion (linearization) at the clean run to estimate the effect per node:

$$\hat{I}_{\text{AtP}}(n; x^{\text{clean}}, x^{\text{noise}}) := (n(x^{\text{noise}}) - n(x^{\text{clean}})) \tau \frac{\partial \mathcal{L}(\mathcal{M}(x^{\text{clean}}))}{\partial n} \Big|_{n=n(x^{\text{clean}})}$$

- Per prompt pair:
 - 1 forward on the clean prompt (cache clean activations and L)
 - 1 backward on the clean prompt (get $\partial L / \partial n$ for all nodes at once)
 - 1 forward on the corrupted prompt (get $n(\text{noise})$ for all nodes)
- Aggregate across the prompt-pair distribution by averaging absolute values (to reduce cross-pair cancellation):

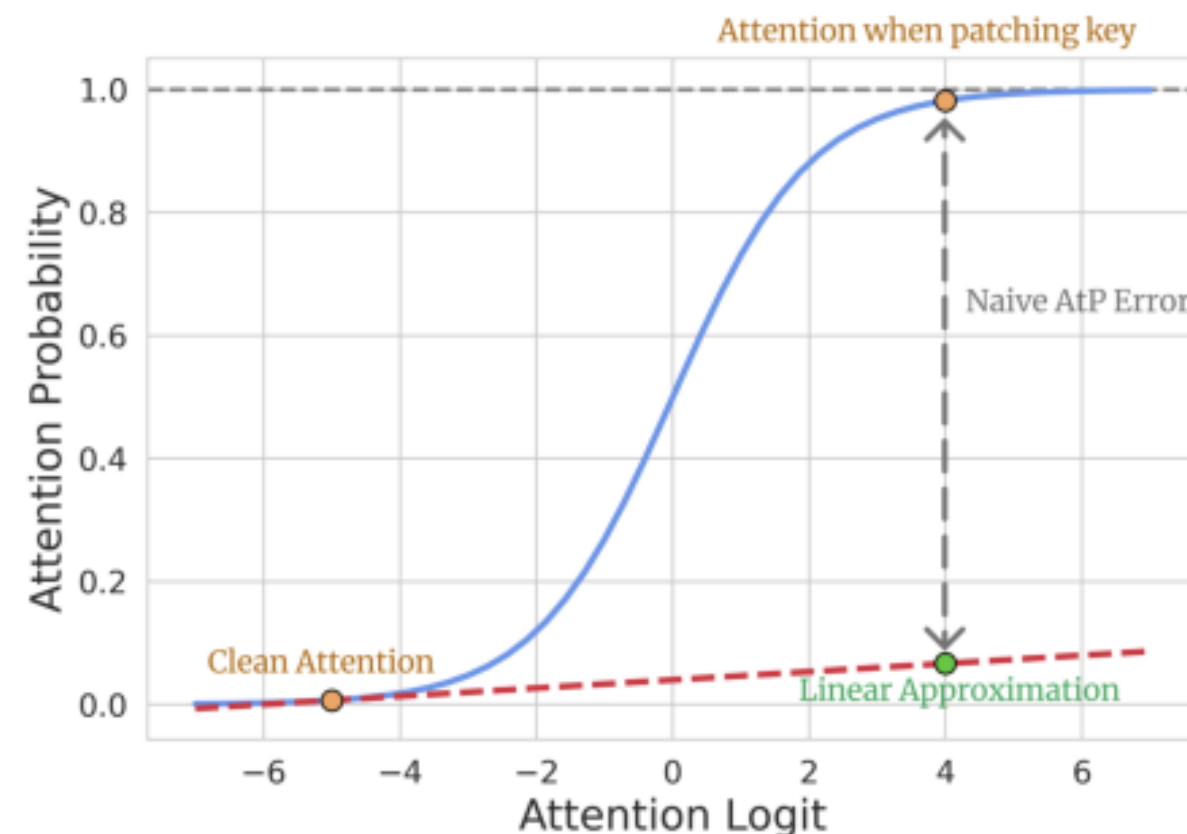
$$\hat{C}_{\text{AtP}}(n) := \mathbb{E}_{x^{\text{clean}}, x^{\text{noise}}} \left[\left| \hat{I}_{\text{AtP}}(n; x^{\text{clean}}, x^{\text{noise}}) \right| \right].$$

Attribution Patching (AtP)

- AtP gives a score for every node from a small, fixed number of passes per prompt pair (does not scale with $|N|$)
- Strategy: use AtP to rank nodes, then verify only the top-K with exact activation patching
- A huge cost reduction with good fidelity for ranking important nodes

AtP limitation: non-linearities

- The softmax is highly non-linear; near-saturated regions (very high or very low probabilities) have near-zero local gradients
- AtP's linearization uses that local gradient; it can severely understate the true effect when you patch queries/keys
- Result: false negatives (AtP says “small effect” when the real patching effect is large)



QK fix

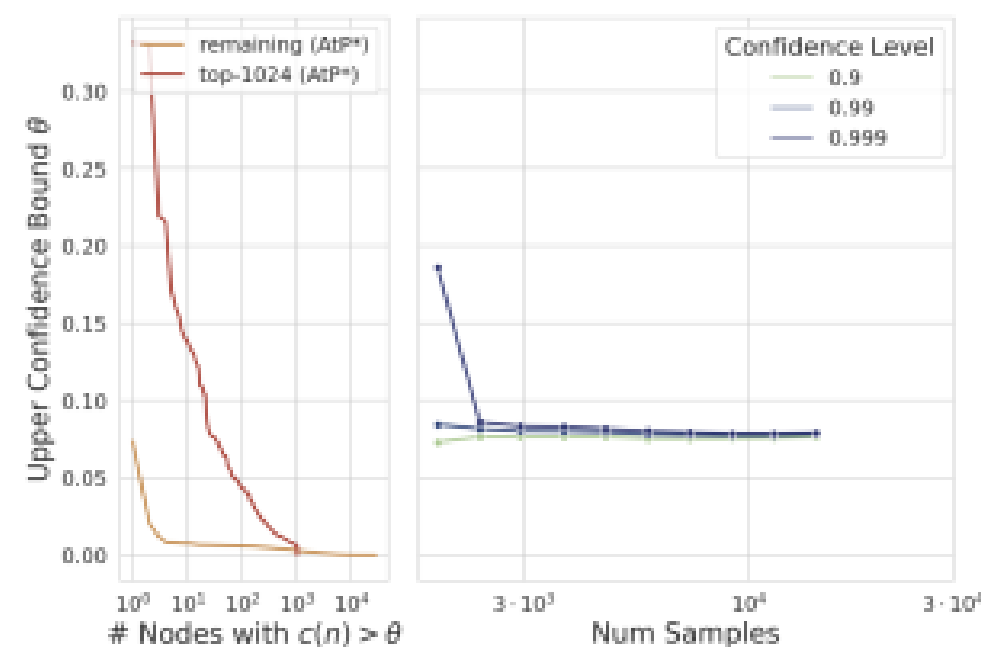
- Recompute attention weights exactly after patching the query/key (using cached clean keys and noise queries, or vice versa)
- Then propagate using gradients from attention weights onward
- Cost: cheaper than re-running two full transformer passes; only recomputes the attention block
- Still scalable!

AtP limitation: effects cancellation

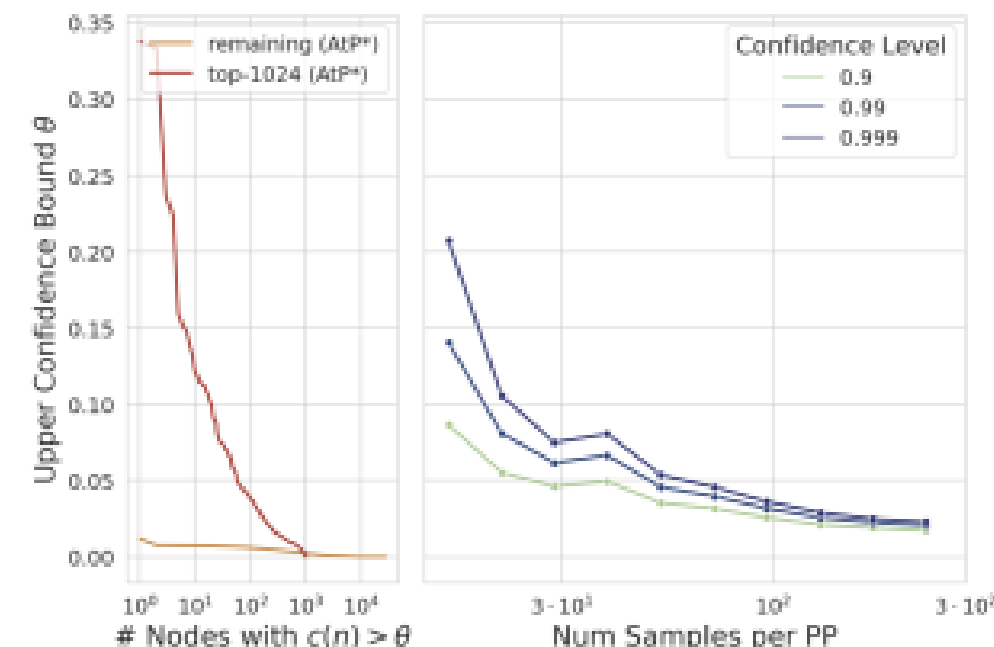
- A node's total effect is the sum of both direct and indirect routes
- Like it happens with the attention softmax, approximation errors in indirect route terms might happen due to non-linearities; if that's the case, it can be increased/decreased and cancelled with direct terms
- As a result, AtP underestimates component effects → false negatives!
- **Fix:** GradDrop
 - How it works: For each layer, zero out gradients (including when propagating to earlier layers), computes AtP under each drop, averages absolute values across layers, and rescales to keep direct-path strength
 - Breaks brittle cancellations between a node's direct and indirect effects, recovering hidden important nodes (especially on single prompt pairs)
 - Cost: adds noticeable amount of extra backward passes per prompt pair, but still is far cheaper than brute-force patching node-by-node

Diagnostics - how much are we missing?

- Statistically bound the largest missed node outside the AtP* top-K
- Subset sampling: patch random subsets of nodes with inclusion probability p on random prompt pairs; record the mean effect when a node is included vs. excluded ($\bar{i} \pm$), plus standard deviations and counts; apply a one-sided Welch's t-test
- Outcome: upper bound on the biggest remaining false negative as a function of sampling budget; much cheaper than exhaustive verification



(a) I0I-PP



(b) I0I

Experimental setup

- Tasks:
 - CITY-PP: Single-pair factual recall (neurons dominate).
 - IOI-PP: Single-pair indirect object identification (attention dominates).
 - RAND-PP: Single random prompt pair from The Pile (sanity check for generalization).
- IOI distribution: 120 prompt pairs over 6 names.
- A-AN distribution: 100 prompt pairs requiring “a” vs “an”.
- Metric: Negative log-probability of the clean target token. Also tested logit difference and log-odds; denoising generally looked worse on IOI.

Key results

- AtP substantially outperforms baseline alternatives (subsampling, blocks, hierarchical), except interactive
- AtP* fixes provide further significant improvement by reducing false negatives:
 - QK fix: major improvement on attention nodes (e.g., IOI-PP).
 - GradDrop: although it recovers neuron false negatives on single prompt pairs (CITY-PP), averaging across pairs reduces its benefit relative to its cost.
- Attention nodes for attention-centric tasks; neuron nodes for MLP-centric
 - AtP* when analysing attention nodes; AtP for neuron nodes
- Neurons may not be semantically monosemantic; SAEs can help!